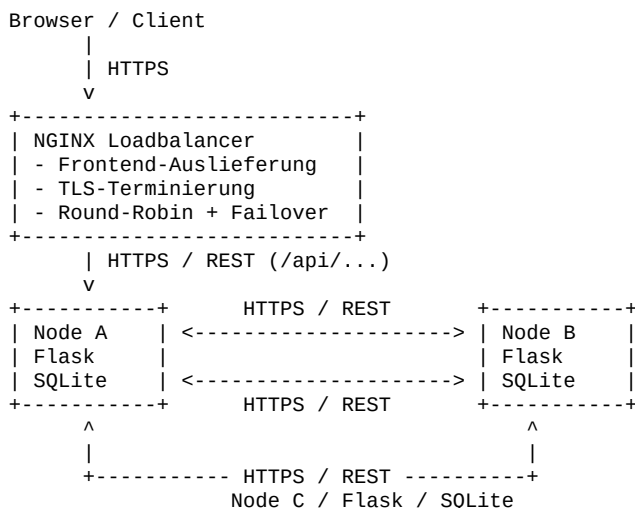


Architektur-Kurzbeschreibung - Verteiltes Commandcenter

Das System ist ein verteilter Statusserver fuer textbasierte Statusmeldungen mit Geokoordinaten. Benutzer greifen ueber eine zentrale HTTPS-URL auf das Frontend zu. Im Backend laufen drei gleichwertige Statusnodes, die Statusobjekte lokal speichern und untereinander replizieren.

Blockdiagramm



Komponenten

Komponente	Aufgabe
Frontend	Weboberflaeche mit Formular, globalem Feed und Leaflet-Karte
NGINX Loadbalancer	Single Point of Access, HTTPS, statisches Frontend, API-Failover
StatusNode A/B/C	CRUD-API, lokale SQLite-Persistenz, Replikation, Bootstrap
SQLite pro Node	Persistenter lokaler Speicher ohne gemeinsame Datenbank

Kommunikation und Sicherheit

Der Client spricht ausschliesslich den Loadbalancer ueber HTTPS an. NGINX verteilt /api/... Requests per HTTPS auf die Statusnodes. Die Nodes replizieren Schreiboperationen per POST /replicate und holen beim Neustart Snapshots per GET /internal/snapshot, ebenfalls ueber HTTPS. Es wird ein selbstsigniertes Dev-Zertifikat verwendet; die interne Zertifikatspruefung ist im Compose-Setup deaktiviert, die Verbindung bleibt aber verschluesselt.

Replikation und Konsistenz

Ein Statusobjekt wird ueber username identifiziert und enthaelt statustext, uhrzeit, latitude, longitude, deleted und originNode. Schreibt ein Client auf eine Node, speichert diese lokal und pusht das Update an ihre Peers. Konflikte werden deterministisch mit Last-Writer-Wins geloest: neuere uhrzeit gewinnt, bei gleicher Zeit entscheidet originNode. Deletes werden als Tombstones repliziert, damit alte Updates gelöschte Einträge nicht wiederherstellen.

Fehlertoleranz

Faellt eine Statusnode aus, leitet NGINX Requests automatisch auf die verbleibenden Nodes weiter. Fehlgeschlagene Peer-Replikationen werden in einer Retry-Queue gespeichert und spaeter nachgeliefert. Startet eine Node neu, befindet sie sich in einer Grace Period, zieht Peer-Snapshots, merged per Last-Writer-Wins und beantwortet erst danach Client-Anfragen.

Start und Demo

Start mit docker compose up --build. Die Anwendung ist danach unter https://localhost:8443 erreichbar. Bei lokal gesetzter .env kann der Port abweichen. Das selbstsignierte Zertifikat muss im Browser akzeptiert werden.

Abgaberelevante Dateien: Dockerfile, docker-compose.yml, README.md, backend/, frontend/, loadbalancer/, docs/.